

Pre-Render Filtering of Unstable Gaussians for Improved Extreme View Synthesis
in 3D Gaussian Splatting

by

Sebastian Teuttli

An Honors Thesis Presented in Partial Fulfillment
of the Requirements for Graduation from Barrett,
The Honors College at Arizona State University

Approved April 2026 by the
Graduate Supervisory Committee:

Dr. Heni Ben Amor, Chair
Dr. Mohamed El Mistiri
Alper Gel

ARIZONA STATE UNIVERSITY

May 2026

ABSTRACT

3D Gaussian Splatting has become a popular method of achieving photo-realistic renders from novel viewpoints while maintaining real-time rendering speeds. Despite its efficiency, 3D Gaussian Splatting struggles with rendering out-of-distribution camera poses, which often results in noisy renders.

This thesis addresses this limitation by introducing an offline preprocessing stage, inspired by EV3DGS. This method evaluates Gaussian stability, implemented as a custom CUDA kernel within the Gsplat framework, across many sampled camera positions. Cameras are sampled from a PCA-aligned ellipsoid fitted to the scene, ensuring robust coverage.

Through pruning Gaussians prior to rendering, the method improves the stability of renders from out-of-distribution views, while preserving the real-time performance advantages of 3D Gaussian Splatting.

Code can be found on **ood_filter** branch at <https://github.com/SebastianTeu/gsplat>

To everyone who has encouraged and guided me throughout this thesis, thank you.

TABLE OF CONTENTS

	Page
LIST OF TABLES	v
LIST OF FIGURES	vi
CHAPTER	
1 Introduction	1
1.1 Neural Rendering and Novel View Synthesis	1
1.2 Thesis Objectives	2
2 Related Work	3
3 Background	6
3.1 3D Gaussian Splatting	6
3.2 Gsplat	8
3.3 EV3DGS	9
4 Thesis Contributions	13
5 Methodology	16
5.1 PCA-Aligned Ellipsoid Fitting	16
5.2 Camera Sampling	17
5.3 View Matrix Construction	18
5.4 Ray-Gaussian Intersection and Instability Scoring	19
5.4.1 Ray-Gaussian Intersection	19
5.4.2 Instability Scoring	21
5.5 Complexity Analysis	24
6 Results	25
6.1 Time Complexity Assessment	29
6.2 Ablation	30
7 Conclusion	32

7.1 Future Work	33
REFERENCES	34
APPENDIX	35
A Full Ablation Results	36

LIST OF TABLES

Table	Page
6.1 Performance comparison of unfiltered and filtered scenes from the Deep Blending dataset [Hedman <i>et al.</i> (2018)]	26
6.2 Selected runs from ablation study over filtering parameters on the CreepyAttic scene. All runs use $n_x = n_y = 100$ and PCA percentile = 0.9.	31
A.1 Full 16 runs of the ablation study over filtering parameters on the CreepyAttic scene. All runs use $n_x = n_y = 100$ and PCA percentile = 0.9.	36

LIST OF FIGURES

Figure		Page
2.1	Overview of GQN for Neural Rendering from Eslami and Rezende (2018)	3
2.2	Overview of NeRF rendering pipeline from Mildenhall <i>et al.</i> (2020)	4
2.3	Overview of Plenoxel model from Sara Fridovich-Keil and Alex Yu <i>et al.</i> (2022)	5
3.1	Gaussian plots in 1-3 dimensions	6
3.2	3D Gaussian Splatting example render from Kerbl <i>et al.</i> (2023)	6
3.3	Overview of 3D Gaussian Splatting model from Kerbl <i>et al.</i> (2023)	7
3.4	Example of an out-of-distribution camera pose. Reproduced from Bow- ness and Poullis (2025)	9
3.5	Noisy render from an indoor scene.	9
4.1	Example of camera sampling using 3 slices and 5 cameras per slice	15
6.1	CreepyAttic before filter (left) and after filter (right). A total of 276,067 out of 4,139,537 Gaussians were removed.	27
6.2	DrJohnson before filter (left) and after filter (right). A total of 191,626 out of 3,775,155 Gaussians were removed.	27
6.3	Playroom before filter (left) and after filter (right). A total of 179,399 out of 2,787,141 Gaussians were removed.	28
6.4	Bedroom before filter (left) and after filter (right). A total of 139,486 out of 799,249 Gaussians were removed.	28
6.5	Run-time of filter on CreepyAttic scenes with varying number of Gaus- sians. Results indicate linear relationship between run-time and scene size.	29

Chapter 1

INTRODUCTION

1.1 Neural Rendering and Novel View Synthesis

Neural rendering has become an important area of research in computer vision, graphics, and deep learning. Neural rendering methods attempt to learn a 3D representation of a scene from a set of images. Once this representation is learned, it can then be used for **novel view synthesis**, where images are generated from viewpoints not included in the original data.

Novel view synthesis is becoming increasingly desirable as it has many real-world useful applications, such as virtual reality, robotics, and disaster response. In emergency situations being able to turn 2D imagery into interactable 3D models that increase spatial understanding for responders allows for faster and more effective planning.

In recent years researchers have proposed many methods for novel view synthesis, and among those, 3D Gaussian Splatting has stood out for its efficiency. This approach represents a scene with many 3D Gaussians. However, when rendering novel views from poses significantly far away (out-of-distribution) from the poses the scene was created with, the results tend to become noisy and unstable. This thesis addresses this issue by incorporating a preprocessing stage that will remove unstable Gaussians from the scene and save the filtered Gaussians for rendering, therefore not affecting the real-time performance of the renderer.

1.2 Thesis Objectives

The primary objectives for the thesis are:

1. **Improve out-of-distribution render quality:** implement a method that reduces noise and instability in 3D Gaussian Splatting renders when viewed from camera poses outside the training distribution.
2. **Preserve real-time performance:** perform the instability filtering prior to rendering in an offline preprocessing stage so that real-time performance is not degraded.
3. **Robust camera sampling:** design a camera sampling strategy that will adapt to the scene to ensure effective coverage with sampled cameras.
4. **GPU acceleration:** implement the metric used for instability calculations within a Gsplat CUDA kernel to keep filtering times practical even for scenes with millions of Gaussians.

RELATED WORK

Initial research focused on representing scenes using neural networks which were conditioned on viewpoint. One example is the Generative Query Network (GQN) shown in Figure 2.1. This approach separates the model into two main components: a representation network and a generation network [Eslami *et al.* (2018)].

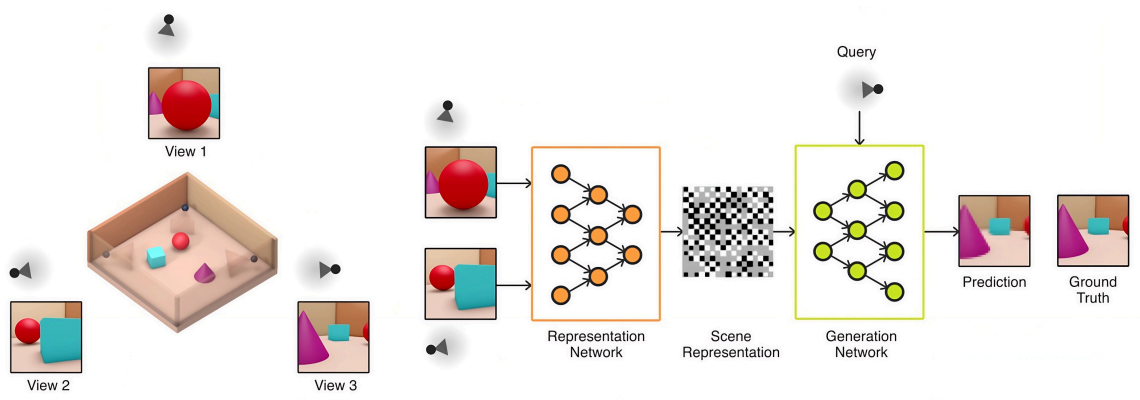


Figure 2.1: Overview of GQN for Neural Rendering from Eslami and Rezende (2018)

The representation network encodes information about the 3D scene from a set of images into a latent vector. This is called **implicit representation**. In implicit representation, the scene is captured within the parameters or latent variables of a neural network, rather than the explicit geometry of the scene. The generation network then uses this representation and a viewpoint (the query) to render an image. These results demonstrated that neural networks could learn enough information about a scene to render an image from novel viewpoints, but it was heavily limited in both resolution and the complexity of the scene.

An influential development in neural rendering came with the introduction of **Neural Radiance Fields (NeRF)**, which greatly improved on the resolution and complexity weaknesses that GQN faced.

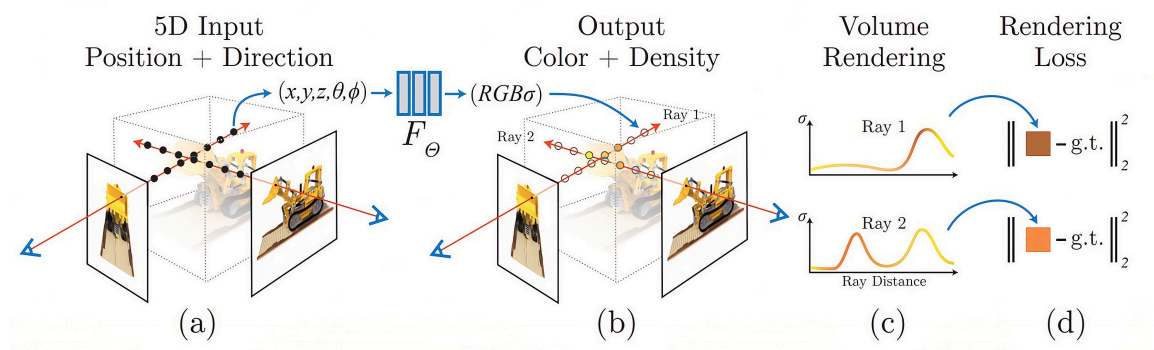


Figure 2.2: Overview of NeRF rendering pipeline from Mildenhall *et al.* (2020)

Similar to GQN, NeRF also uses implicit representation. However, instead of using a neural network to encode the scene as a latent vector, NeRF learns the scene as a continuous function, meaning it can be evaluated at any point in space and with any viewing direction. An overview of the NeRF rendering pipeline is shown in Figure 2.2. The network is fed a 3D location and viewing direction as input and outputs a transparency and color [Mildenhall *et al.* (2020)]. This mapping is learned at training time from the set of training images.

When rendering a new image, a ray is cast from the camera through each pixel into the scene. Points along each ray are then sampled and combined to determine the final color of the pixel. In the training loop, the model attempts to render one of the images from the dataset and compares it with the ground truth photo. The model then updates its parameters with gradient descent to minimize the difference between the two. The model eventually converges to a representation that recreates the training views and also allows for the synthesis of new viewpoints.

NeRF could produce very detailed and realistic images. However, NeRF also has a significant limitation in practice. Due to being stored as an implicit representation,

NeRF renderings require querying the network many times for each ray. Since a ray is cast for each pixel, to render just one high-quality image could require thousands of queries to the network. This makes using NeRF for real-time rendering impractical.

Due to this computational cost, researchers have explored alternative solutions for scene representation that are as detailed and realistic as NeRF but also cheaper to render. One such category of alternatives that has emerged use **explicit scene representation**. Instead of representing the scene within a neural network, the scene itself is represented using discrete geometric or volumetric structures. Plenoxels is one example of this, where the scene is represented as a grid of voxels, where each voxel stores spherical harmonic coefficients for color [Sara Fridovich-Keil and Alex Yu *et al.* (2022)], as shown in Figure 2.3. Since this representation is explicit rather than encoded within a neural function, it can be rendered using more traditional rendering techniques.

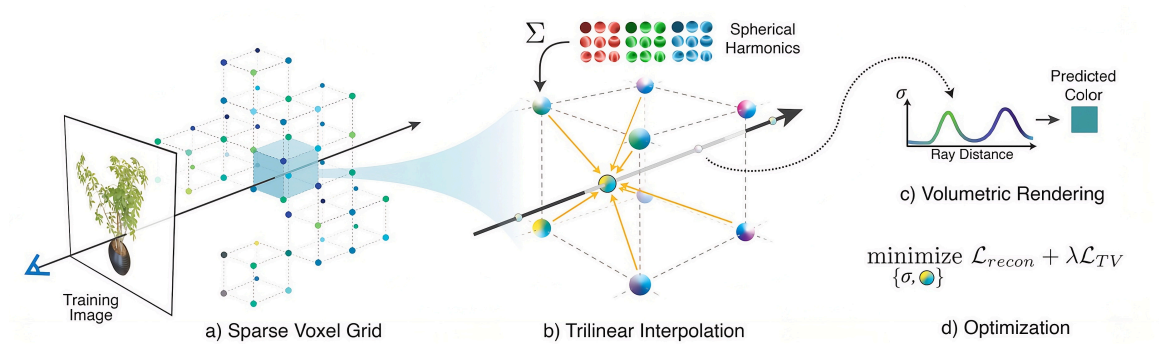


Figure 2.3: Overview of Plenoxel model from Sara Fridovich-Keil and Alex Yu *et al.* (2022)

More recently, **3D Gaussian Splatting** has emerged as an extremely efficient alternative for explicit scene representation which allows for fast real-time neural rendering.

Chapter 3

BACKGROUND

3.1 3D Gaussian Splatting

In 3D Gaussian Splatting, the scene is represented as a collection of 3D ellipsoids, called Gaussian splats [Kerbl *et al.* (2023)]. These Gaussian splats are 3D Gaussians with a position, opacity, shape, size, and color. Figure 3.1 illustrates this approach.

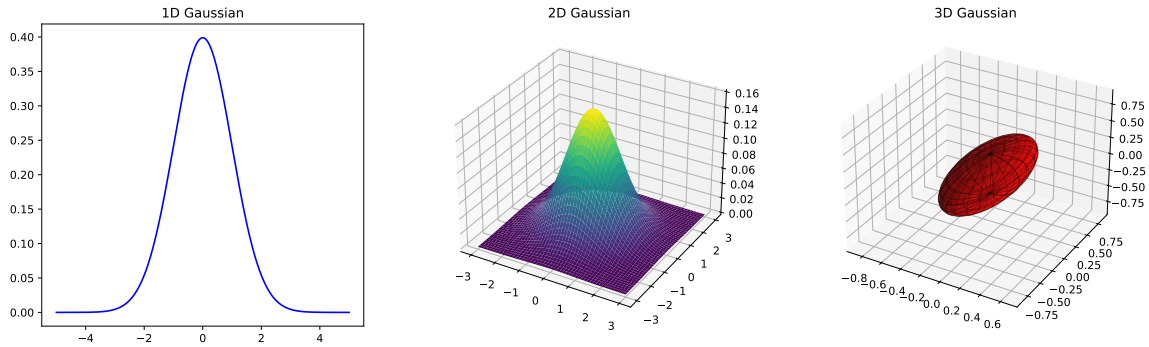


Figure 3.1: Gaussian plots in 1-3 dimensions

With a sufficient number of these Gaussian splats, the scene becomes photorealistic, as shown in Figure 3.2. Since the Gaussian’s properties are a literal representation of the scene, once obtained, the rendering becomes much more computationally efficient than NeRF. The pipeline to obtain the Gaussians can be seen in Figure 3.3.



Figure 3.2: 3D Gaussian Splatting example render from Kerbl *et al.* (2023)

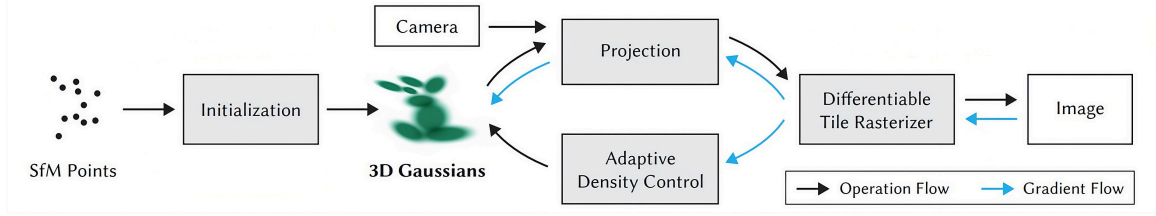


Figure 3.3: Overview of 3D Gaussian Splatting model from Kerbl *et al.* (2023)

To initialize the Gaussian placements, the method uses a point cloud generated via structure-from-motion (SfM), a technique that reconstructs the general structure of a 3D scene and camera poses from a set of 2D images. A commonly used program for SfM is COLMAP [Schönberger and Frahm (2016)].

Once initialized, the Gaussians enter a training loop similar to NeRF. The current Gaussians are used to render an image from the data, and gradient descent is then used to minimize the difference. Additionally, the pipeline will split large Gaussians, clone Gaussians in high-error regions, and prune Gaussians with low opacity. This process is called **adaptive density control**.

Rendering is optimized using projection and tiling. During projection, the 3D position, size, and orientation of each Gaussian is mapped into the camera’s local 2D space. From the camera’s perspective, the now 2D ellipses have the same appearance as the original 3D ellipsoids. Tiling then divides the screen into small regions, and each projected Gaussian is assigned to the tiles that its 2D projection overlaps. During rendering, computations are only performed with the Gaussians associated with a given tile, reducing the total number of computations. This tiling technique is a type of rasterization.

A 3D Gaussian $\mathcal{G}$ is parametrized by a mean μ , covariance matrix Σ , opacity α , and color c . The mean and covariance matrix determine the Gaussian’s shape, orientation, and size:

$$\mathcal{G}(x) = \exp\left(-\frac{1}{2}(x - \mu)^T \Sigma^{-1}(x - \mu)\right) \quad (3.1)$$

The covariance matrix can be defined by a scaling matrix and rotation matrix, where the scaling matrix S defines the axis lengths of the 3D ellipsoid, and the rotation matrix R defines how that ellipsoid is oriented in space.

$$\Sigma = RSS^T R^T \quad (3.2)$$

Parameterizing the covariance matrix in this manner ensures it remains positive semi-definite, guaranteeing that the Gaussian has non-negative variance in all directions.

Following the projection and tiling step, the color C for pixel x is determined by alpha compositing the N Gaussians that contribute to that pixel.

$$C(x) = \sum_{i=1}^N c_i \alpha_i \mathcal{G}_i(x_i) \prod_{j=1}^{i-1} (1 - \alpha_j \mathcal{G}_j(x_j)) \quad (3.3)$$

This process is then performed for each pixel in the image to produce the final render.

3.2 Gsplat

Gsplat is an open-source library that implements 3D Gaussian Splatting with highly optimized CUDA kernels [Ye *et al.* (2024)]. Gsplat’s implementation was able to achieve up to 10% faster training times and 4x lower memory usage compared to the original implementation.

3.3 EV3DGS

While 3D Gaussian Splatting produces highly realistic renderings, it notably struggles with novel view synthesis from camera positions significantly outside the distribution from those the scene was trained on. An example of one such pose is shown in Figure 3.4. Since these viewpoints are significantly different than those seen during training, the model tends to produce unstable or noisy predictions that obscure the main scene, shown in Figure 3.5.

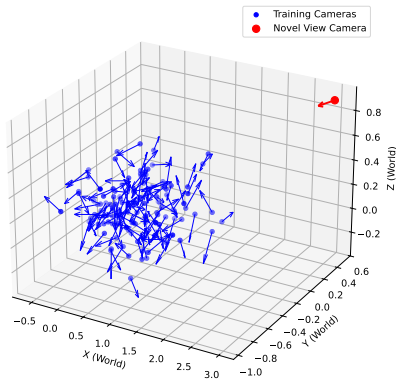


Figure 3.4: Example of an out-of-distribution camera pose. Reproduced from Bowness and Poullis (2025)

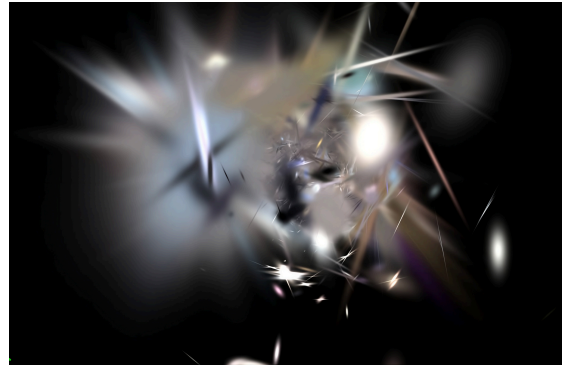


Figure 3.5: Noisy render from an indoor scene.

To address this limitation, EV3DGS introduces a method of filtering out these noisy Gaussians [Bowness and Poullis (2025)]. For each pixel, a ray is cast into the scene and each Gaussian it intersects is assigned a sensitivity score, which measures how geometrically unstable that Gaussian is at the point of intersection under small spatial perturbations. If this score is higher than a given threshold, the Gaussian is marked unstable. On a second pass, they calculate each Gaussian’s rejection ratio, which is the ratio of the number of times it is marked unstable to the total number of times it was intersected. If this ratio is higher than a given threshold, the Gaussian is skipped during rendering.

The rays can be parameterized with an origin o and a direction r , with t being the distance along the ray:

$$x = o + tr \quad (3.4)$$

The sensitivity metric will be evaluated for each of the K Gaussians the ray intersects, at the point where each Gaussian has its peak contribution. To find this point we will redefine the ray in the Gaussian's local coordinate space. In this space, the 3D Gaussian can be evaluated as an isotropic unit Gaussian, i.e., its variance is equal along all axes. This is done by applying the inverse of the scaling matrix and the rotation matrix of the Gaussian, and by repositioning the Gaussian's center μ as the origin:

$$o_g = S_k^{-1} R_k (o - \mu) \quad (3.5)$$

$$r_g = S_k^{-1} R_k r \quad (3.6)$$

The ray in the Gaussian's local space then becomes:

$$x_g = o_g + tr_g \quad (3.7)$$

This full transformation, including scale, is solely used to find t_{min} , which is the depth of peak Gaussian contribution along the ray. The scale term is later dropped when computing the sensitivity metric, for reasons discussed when deriving it.

Since the Gaussian can now be evaluated as an isotropic Gaussian, contribution along any line (including the ray) reduces to a 1D Gaussian:

$$G^{1D}(t) = \exp\left(-\frac{1}{2}x_g^T x_g\right) \quad (3.8)$$

$$= \exp\left(-\frac{1}{2}(o_g + tr_g)^T(o_g + tr_g)\right) \quad (3.9)$$

$$= \exp\left(-\frac{1}{2}(t^2 r_g^T r_g + 2to_g^T r_g + o_g^T o_g)\right) \quad (3.10)$$

The contribution of the Gaussian will be at a maximum when the exponent is largest, which is when the quadratic is smallest. The minimum for a quadratic can be found as $-\frac{b}{2a}$. For the quadratic above, that results in a minimum of:

$$t_{min} = -\frac{o_g^T r_g}{r_g^T r_g} \quad (3.11)$$

The final pixel color can now be calculated using alpha compositing as mentioned before, with each Gaussian's calculated t_{min} being their depth:

$$C(o, r) = \sum_{k=1}^K c_k \alpha_k \mathcal{G}_k^{1D}(t_{k,min}) \prod_{j=1}^{k-1} (1 - \alpha_j \mathcal{G}_j^{1D}(t_{j,min})) \quad (3.12)$$

To find how the color of a pixel changes with small perturbations to the position of an intersected Gaussian, the gradient of the above color function is taken with respect to the 3D position x .

Using $a_i = \alpha_i \mathcal{G}_i(x_i)$:

$$\nabla C = \sum_{k=1}^K c_k a_k \prod_{j=1}^{k-1} (1 - a_j) \left(\sum_{j=1}^{k-1} \frac{a_j \Sigma_j^{-1} x_j}{1 - a_j} - \Sigma_k^{-1} x_k \right) \quad (3.13)$$

For computational efficiency, c_k is set a scalar $c_k = 1$. Since it only affects color scaling, this simplification decouples the sensitivity metric from color, meaning that now only the spatial configuration of Gaussians along the ray (their positions, rotations, accumulated densities) determine the score.

Additionally, in order to isolate rotational instability for a Gaussian, the scale term S_k is dropped from equations (3.5) - (3.7). Including scale transforms the ray

into a space where the Gaussian is isotropic, which loses the shape information of the Gaussian. By only including rotation, the ray is now in a space where the Gaussian maintains its shape, but the ray's components are aligned with the Gaussian's principal axes. This means the sensitivity metric will flag a Gaussian as unstable if its orientation relative to the viewing ray is unstable, rather than flagging it due to its size or scale magnitude.

These two modifications results in the sensitivity metric:

$$S = \sum_{k=1}^K a_k \prod_{j=1}^{k-1} (1 - a_j) \left(\sum_{j=1}^{k-1} \frac{a_j x_j}{1 - a_j} - x_k \right) \quad (3.14)$$

The point of maximum Gaussian contribution along the ray occurs at depth t_{min} , where $x = R_k(o - \mu_k) + tR_k r$ expresses the ray position in the rotation-aligned coordinate system of the Gaussian.

THESIS CONTRIBUTIONS

While EV3DGS is extremely effective at cleaning up the render of noisy Gaussians, the added computational overhead of casting the rays and scoring each intersected Gaussian impairs real-time performance.

This thesis addresses these computational limitations by shifting the filtering to an offline pre-rendering stage. The primary contributions are:

- **Offline Gaussian pruning:** a new pre-render step that will prune Gaussians in a given scene prior to rendering, eliminating the need for costly per-frame ray evaluations during rendering.
- **PCA-aligned ellipsoidal camera sampling:** a robust camera sampling strategy that will conform to a given scene.
- **Gsplat CUDA-based implementation:** an implementation of an EV3DGS-style filter into a Gsplat CUDA kernel, allowing for efficient evaluation of the sensitivity metric across a large number of Gaussians.

While applying the filter as an offline preprocessing step avoids performance loss, it is worth noting this loses the per-view adaptability of the original EV3DGS approach. Gaussians flagged from one view may be stable from another, so permanently removing these Gaussians risks degrading the rendering quality of in-distribution views.

Camera positions are sampled from the surface of an ellipsoid that is fitted to the scene. The axes of this ellipsoid are found through principal component analysis (PCA) on the Gaussian centers. The radii are calculated as a percentile-based extent

of the Gaussians projected onto each principal axis. Using a percentile instead of the most distant Gaussian on each axis makes the calculations more robust to extreme outliers, which may cause the cameras to be sampled too far away, leading to fewer ray-Gaussian intersections.

To allow for further control of the sampling region, the ellipsoid radii can be scaled by a floating point value. In addition, multiple scalars can be provided which enables multiple ellipsoids of various sizes to be used as sampling regions, providing more flexible coverage of the scene. Both the percentile and radii scalars are adjustable hyperparameters.

The 3D ellipsoid is then discretized into slices along its smallest principal axis, and cameras are then uniformly sampled along each slice, looking towards the center of the scene. The camera sampling is shown in Figure 4.1 The specific amount of cameras sampled for each slice is another adjustable hyperparameter.

Utilizing a PCA-aligned ellipsoid for camera sampling rather than a sphere allows the ellipsoid to conform around the shape of the scene. This improves generalization to anisotropic scenes which are dominated by one axis (e.g, a tall building or a long hallway).

Implementing the sensitivity calculations in a CUDA kernel allows parallel evaluation on the GPU, with each thread handling an individual ray. This keeps the preprocessing stage practical, even for scenes with a large number of Gaussians.

Ellipsoid Camera Sampling Normal to Minor Axis

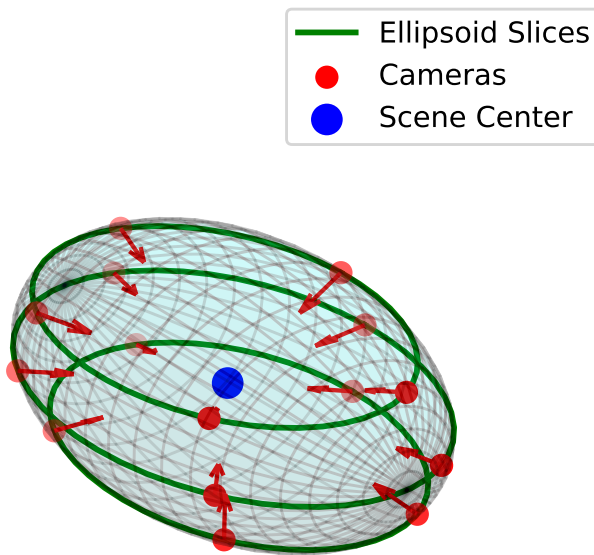


Figure 4.1: Example of camera sampling using 3 slices and 5 cameras per slice

METHODOLOGY

This section details the algorithms used for the offline Gaussian pruning and camera sampling strategies. The main components of the camera sampling pipeline are computing the base PCA-aligned ellipsoid, the scaling and sampling of the ellipsoids, and the construction of the view matrices from the camera positions. The main components of the EV3DGS-style filter are collecting which Gaussians contribute to each ray, and evaluating the sensitivity metric on those Gaussians in order of depth.

5.1 PCA-Aligned Ellipsoid Fitting

The first step is fitting a 3D ellipsoid to approximate the distribution of Gaussians in the scene. The center is taken as the mean of all the Gaussians in the scene, the axes are determined through PCA on the Gaussian centers, and the radii are defined as a user given percentile (e.g, 90th) of the Gaussian centers projected onto each principal axis. This approach is summarized in Algorithm 1.

An earlier method attempted to sample from a sphere centered at the mean, with its radius determined by the farthest Gaussian. This ensures all Gaussians are contained within the sampling volume. In practice this approach was found to be sensitive to outliers and does not adapt well to anisotropic or elongated scenes. In these cases, the volume of the sampling space is unnecessarily large. As a result, many cameras are placed in sparse sections of the scene. This is inefficient for scene coverage. In the PCA-aligned ellipsoidal approach, these issues are addressed by adapting to the shape of the scene and defining its radii using a percentile-based approach. This results in a tighter bounding volume and more useful camera placement.

Algorithm 1: Compute PCA-Aligned Ellipsoid

Input: Gaussian means $\mu \in \mathbb{R}^{N \times 3}$, percentile p

Output: Center $c \in \mathbb{R}^3$, principal axes $V \in \mathbb{R}^{3 \times 3}$, base radii $r \in \mathbb{R}^3$

```
 $c \leftarrow \text{mean}(\mu)$                                      // mean of Gaussian means
 $\mu^{centered} \leftarrow \mu - c$ 
 $U, \Sigma, V^T \leftarrow \text{SVD}(\mu^{centered})$            // rows of  $V^T$  are principal axes,
ordered by decreasing variance
 $P \leftarrow \mu^{centered} V$                              // project means onto each axis
 $r \leftarrow \text{quantile}(|P|, p)$ 
return  $c, V, r$ 
```

5.2 Camera Sampling

Given the fitted ellipsoid, a set of camera positions is sampled from its surface. The positions are obtained by splitting the ellipsoid into L slices with C cameras distributed uniformly around the edge of each slice. The slices at the poles of the ellipsoid are not included as all positions collapse to a single point at the pole which results in many redundant camera positions that do not provide additional scene coverage. As such, $L + 2$ slices are calculated and the poles are then excluded.

To ensure coverage of both near and far viewpoints of the scene, cameras are sampled from multiple scaled versions of the fitted ellipsoid. The number and sizes of these ellipsoids are governed by the set of scalars S . Smaller ellipsoids closer to the scene tend to generate rays with a higher number of collisions and detect a large proportion of the unstable Gaussians. However, if a scene has many unstable floaters far from the main geometry, additional larger ellipsoids can be sampled to effectively prune these as well. Algorithm 2 describes the full sampling procedure.

Algorithm 2: Sample Camera Positions on Scaled Ellipsoids

Input: Center c , axes V , base radii r , ellipsoid scalars $S = \{s_1, s_2, \dots\}$, slices

L , cameras per slice C

Output: Camera positions $P = \{p_1, \dots, p_{L \times C}\}$

$P \leftarrow []$

foreach $s \in S$ **do**

```
     $\hat{r} \leftarrow r \cdot s$  // scale the base radius
    foreach  $l \in \{1, \dots, L\}$  do
         $h_{unscaled} \leftarrow \text{linspace}(-1, 1, L + 2)[1:-1][l]$  a // exclude poles
         $h \leftarrow h_{unscaled} \cdot \hat{r}_{\text{minor}}$  // height in world space
         $\rho \leftarrow \sqrt{1 - h_{unscaled}^2}$  // cross-section scale at this height
        foreach  $c' \in \{1, \dots, C\}$  do
             $\theta \leftarrow 2\pi \frac{c'}{C}$  // angle for this camera on cross-section
             $p \leftarrow c + h v_{\text{minor}} + \rho \hat{r}_{\text{major}} \cos(\theta) v_{\text{major}} + \rho \hat{r}_{\text{mid}} \sin(\theta) v_{\text{mid}}$ 
            append  $p$  to  $P$ 
```

return P

^a $\text{linspace}(a, b, n)$ is a function that generates n evenly spaced values between a and b

5.3 View Matrix Construction

Each sampled position generates a corresponding view matrix using a look-at formulation from Khronos Group (nd), and described in Algorithm 3. The view matrix is constructed to look towards the scene center c to ensure all cameras face inward towards the main geometry.

Algorithm 3: Construct View Matrix (Look-At)

Input: Camera position $p \in \mathbb{R}^3$, scene center $c \in \mathbb{R}^3$, world up vector u

Output: View matrix $M \in \mathbb{R}^{4 \times 4}$

```
 $f \leftarrow \text{normalize}(c - p)$  // forward  
 $r \leftarrow \text{normalize}(f \times u)$  // right  
 $u_{ortho} \leftarrow r \times f$  // corrected up  
 $R \leftarrow [r, u_{ortho}, -f]^T$   
 $t \leftarrow -Rp$   
 $M \leftarrow \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix}$   
return  $M$ 
```

5.4 Ray-Gaussian Intersection and Instability Scoring

With a set of view matrices established, unstable Gaussians are pruned through casting a grid of $n_x \times n_y$ rays through a synthetic camera combined with the instability-based approach introduced in EV3DGS (with a modification detailed later in this section). Each individual ray is handled by a CUDA thread and the process is comprised of two stages: candidate collection and instability scoring.

5.4.1 Ray-Gaussian Intersection

For each ray, all Gaussians that could contribute to that ray are collected and sorted by depth, as described in Algorithm 4. Each Gaussian is evaluated in its own rotation-aligned local coordinate space via transforming the camera origin and ray direction by the rotation matrix derived from each Gaussian’s stored quaternion. Doing so obtains c^{local} and d^{local} , which express the camera position and ray direction

in the rotation-aligned space. The depth value t_i is calculated as the closest point on the ray to the Gaussian center in this local frame. Gaussians with a negative t_i or a t_i less than the near plane get discarded, meaning Gaussians behind the camera or closer than the near plane are rejected as hits. The remaining hits are then sorted by depth in preparation for scoring.

A notable deviation from the methodology EV3DGS proposed is that scale is not incorporated when calculating t_i , a choice made to avoid mixing coordinate systems when calculating hit depth and evaluating instability. Since the sensitivity metric evaluates x_g without scale, calculating t_i with scale would locate the peak contribution for the Gaussian in one coordinate space but the sensitivity metric would use it to evaluate the Gaussian in another. By computing both in the same rotation-only aligned space, it ensures t_i will correctly represent the depth of maximum contribution in the space the metric is evaluated in.

It is worth noting that when attempting to compute t_i in a scale-aligned coordinate system and using it for the sensitivity metric, the filter had noticeably degraded results. This is likely due to the previously stated inconsistency between the space where peak contribution was calculated and the space in which that depth was used.

Algorithm 4: Ray-Gaussian Intersection (One Thread per Ray)

Input: Gaussians $\{(\mu_i, q_i)\}_{i=1}^N$, camera position (ray origin) p_{cam} , ray

direction d , near plane d_{near}

Output: Sorted hit list $H = \{(t_i, i)\}$

$H \leftarrow []$

for $i \leftarrow 1$ **to** N **do**

$R_i \leftarrow \text{quat_to_rotmat}(q_i)$ ^a

$c^{local} \leftarrow R_i^T(p_{cam} - \mu_i)$

$d^{local} \leftarrow R_i^T d$

$t_i = -\frac{d^{local} \cdot c^{local}}{d^{local} \cdot d^{local}}$ // Eq. 3.11

if $t_i \leq 0$ **or** $t_i \leq d_{near}$ **then continue**

 append (t_i, i) to H

return sort(H , key= t)

^aquat_to_rotmat() is a function that converts quaternions to a rotation matrix.

5.4.2 Instability Scoring

The sorted hits are then iterated front-to-back to compute the instability score for each Gaussian along the ray, as shown in Algorithm 5. Two quantities are maintained throughout the process: a transmittance value T which is initialized to one, and a running ray centroid $\bar{x}$ which is initialized to zero. The transmittance value represents the fraction of light along the ray that is not yet occluded, and the ray centroid represents the opacity-weighted mean hit positions of all stable Gaussians accumulated thus far.

For each candidate Gaussian, the opacity α at the hit point x_g is calculated by evaluating the Gaussian at the hit point in the rotation aligned space and multiplying

by the Gaussians base opacity o_i . Gaussians with an $\alpha < \frac{1}{255}$ are considered negligible and are skipped.

The sensitivity value is then calculated as $\|\alpha \cdot T \cdot (\bar{x} - x_g)\|^2$. Intuitively, this term measures how much the current ray centroid $\bar{x}$ would change if this Gaussian were to be included in the render. If the change is large, it indicates the Gaussian lies far from the current ray centroid and has sufficient opacity and transmittance to meaningfully shift it, which is an indicator for a far away opaque floater. The transmittance value T acts as a weight that will penalize Gaussians early in the ray traversal more heavily for displacement, as these typically have greater visual impact on the render.

If the sensitivity value is greater than a user-given threshold τ_{xg} , the Gaussian is marked as unstable. Importantly, unstable Gaussians do not update either $\bar{x}$ or T . This ensures an unstable Gaussian does not prevent the Gaussians behind it from properly being evaluated. If a Gaussian is deemed stable, $\bar{x}$ and T are updated accordingly. Traversal along the ray will proceed until there is nearly no remaining light along the ray ($T < 10^{-4}$).

After all cameras and rays have been processed, a final rejection ratio is calculated for each Gaussian as $\frac{reject_counts[i]}{total_count[i]}$. If a Gaussian’s rejection ratio is greater than a user-given threshold τ_{ratio} , the Gaussian is considered unstable and is pruned from the scene.

Algorithm 5: Instability Scoring (One Thread per Ray)

Input: Sorted hit list H , Gaussians $\{(q_i, o_i)_1, \dots (q_i, o_i)_N\}$, camera position

(ray origin) p_{cam} , sensitivity threshold τ_{xg}

Output: $reject_counts[N]$, $total_counts[N]$ // updated in-place

$T \leftarrow 1$

$\bar{x} \leftarrow 0$

foreach $(t, i) \in H$ **do**

$R_i \leftarrow \text{quat_to_rotmat}(q_i)$

$c^{local} \leftarrow R_i^T(p_{cam} - \mu_i)$

$d^{local} \leftarrow R_i^T d$

$x_g \leftarrow c^{local} + td^{local}$

$\alpha \leftarrow \min(0.99, o_i \cdot \exp(-\frac{1}{2}\|x_g\|^2))$

if $\alpha < \frac{1}{255}$ **then continue**

$S_{score} \leftarrow \|\alpha \cdot T \cdot (\bar{x} - x_g)\|^2$ // Eq. 3.14

$total_counts[i]++$

if $\tau_{xg} > 0$ **and** $S_{score} > \tau_{xg}$ **then**

$reject_counts[i]++$

else

$\bar{x} \leftarrow \bar{x} + \frac{\alpha}{1-\alpha} x_g$

$T \leftarrow T \cdot (1 - \alpha)$

if $T < 10^{-4}$ **then break**

An attempt was made to cache hit data for c^{local} and d^{local} in GPU memory from Algorithm 4 as to avoid recomputation in Algorithm 5. However, this resulted in worse performance. A likely explanation for this is that the increased memory overhead

resulted in additional accesses to higher level memory, making the process overall slower than just performing the recomputation.

5.5 Complexity Analysis

With N number of Gaussians in the scene, ray grid dimensions $n_x = n_y = 100$, ellipsoid scalars S , number of slices L , and number of cameras per slice C . The total number of synthetic cameras is $M = |S| \times L \times C$.

PCA Ellipsoid: To calculate a PCA-aligned ellipsoid involves calculating the mean of all Gaussian centers, performing a subtraction on all Gaussian centers, performing an SVD, and a percentile computation over the Gaussian centers, each of these is $O(N)$.

Camera Sampling: Sampling camera positions on the surface of the ellipsoid involves iterating through each combination of scalar, slice, and camera per slice, which is $O(|S| \times L \times C) = O(M)$. This is often negligible compared to the filter cost.

CUDA Filter: The majority of the computational cost comes from the instability evaluation. For each of the M cameras, a CUDA kernel casts $n_x \times n_y$ rays in parallel, and each ray iterates over the N Gaussians to collect the ray-Gaussian intersections. The total complexity for each camera is then $O(N \times n_x \times n_y)$. The overall time complexity of the filter is then:

$$O(M \times N \times n_x \times n_y) = O(|S| \times L \times C \times N \times n_x \times n_y) \quad (5.1)$$

With fixed parameters, the filters run-time scales linearly with N .

RESULTS

The proposed methods are tested using renders generated from Gsplat’s simple trainer with the default optimization strategy, trained for 30,000 iterations. Unless otherwise noted, no image down-scaling is applied. The sparse point clouds are generated using COLMAP. Experiments are primarily conducted on the Deep Blending dataset [Hedman *et al.* (2018)], and run on an NVIDIA RTX 5070 GPU.

In addition, a custom-captured indoor scene of a master bedroom is included to evaluate performance on real-world data. The scene data is captured on an iPhone 13 Pro Max and is trained using the same pipeline, but with the images down-scaled by a factor of 4 for computational constraints.

All filtered scenes were run with the same hyperparameters of sensitivity threshold $\tau_{xg} = 0.0001$, ratio threshold $\tau_{ratio} = 0.25$, $n_x = n_y = 100$, percentile $p = 90$, ellipsoid scalars $S = \{2.0, 4.0\}$, number of slices $L = 5$, and number of cameras per slice $C = 6$. These hyperparameters were found to remove a large majority of the noisy Gaussians in the scene while keeping the pre-render step fast and not over-pruning the scene of useful detail. As such, they’ve been chosen as the default parameters for the filter.

Since this filter is used to improve out-of-distribution camera pose synthesis, it is not appropriate to use the poses from the training set to evaluate the filter. As such, evaluations will be measured using no-reference image quality metrics. The specific metrics are:

- **Natural Image Quality Evaluator (NIQE):** evaluates how much an image deviates from the statistical regularities learned from high-quality natural images [Mittal *et al.* (2013)].

- **Blind/Referenceless Image Spatial Quality Evaluator (BRISQUE):** evaluates an image using a model trained on images with human-assigned distortion scores [Mittal *et al.* (2012)].
- **Perception-based Image Quality Evaluator (PIQE):** evaluates an image based on perceived distortion through block-wise analysis of blocking artifacts and noise [N *et al.* (2015)].

Rendered images are taken from an orbit of views around the averaged center of all Gaussians, with its orbit radius set to 5 times the 90th percentile distance of all Gaussian means from the averaged center. This gives an out-of-distribution pose and ensures the scene remains in frame.

The FPS is calculated by timing the response of 100 rasterization calls along with a 10 frame warm-up.

The rendered images are trimmed to the smallest rectangle that includes all the Gaussians in the scene. For all metrics a lower score is better. Additionally, the mean FPS for each scene before and after the filter will be evaluated to assess real-time performance.

Table 6.1: Performance comparison of unfiltered and filtered scenes from the Deep Blending dataset [Hedman *et al.* (2018)]

Scene	No Filter				Filter				Filter Time (sec)
	NQ	BR	PQ	FPS	NQ	BR	PQ	FPS	
CreepyAttic	6.07	64.03	55.79	58.83	3.82	30.95	42.19	60.46	105.292
DrJohnson	6.90	86.76	64.32	69.48	4.22	34.75	50.12	62.55	101.414
Playroom	7.98	80.80	70.15	87.83	4.57	42.76	50.60	89.34	92.065
Bedroom	7.41	69.15	76.14	263.61	4.94	38.18	59.67	262.74	52.684

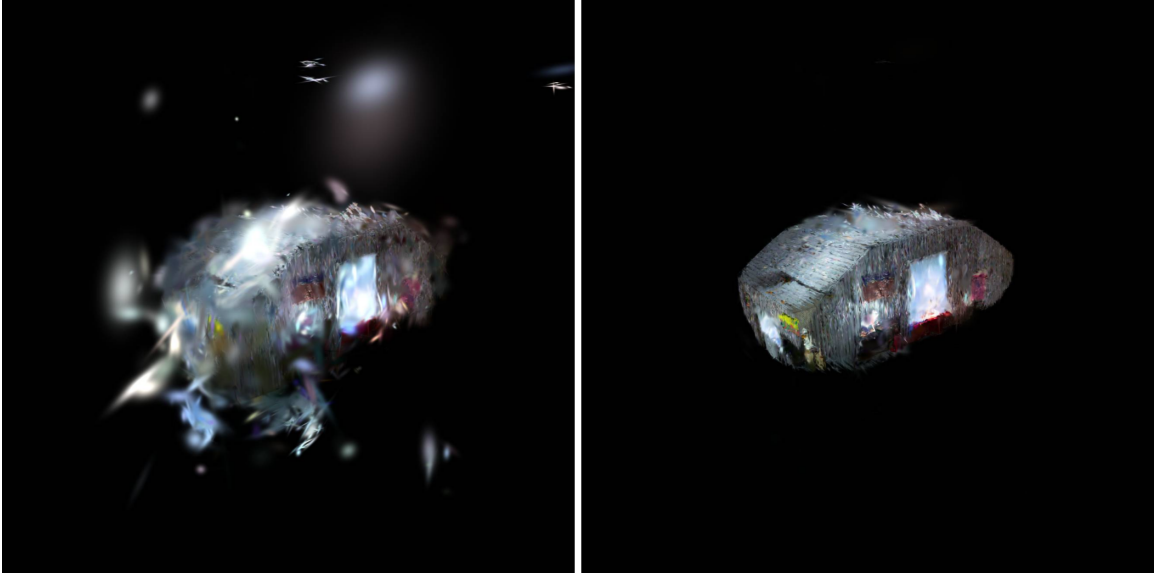


Figure 6.1: CreepyAttic before filter (left) and after filter (right). A total of 276,067 out of 4,139,537 Gaussians were removed.

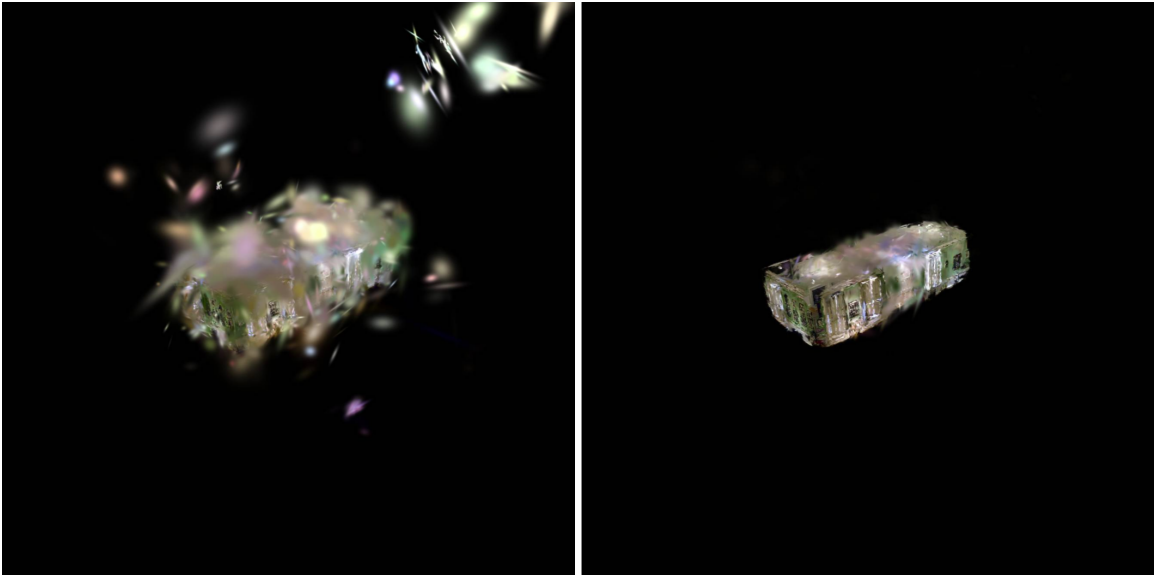


Figure 6.2: DrJohnson before filter (left) and after filter (right). A total of 191,626 out of 3,775,155 Gaussians were removed.

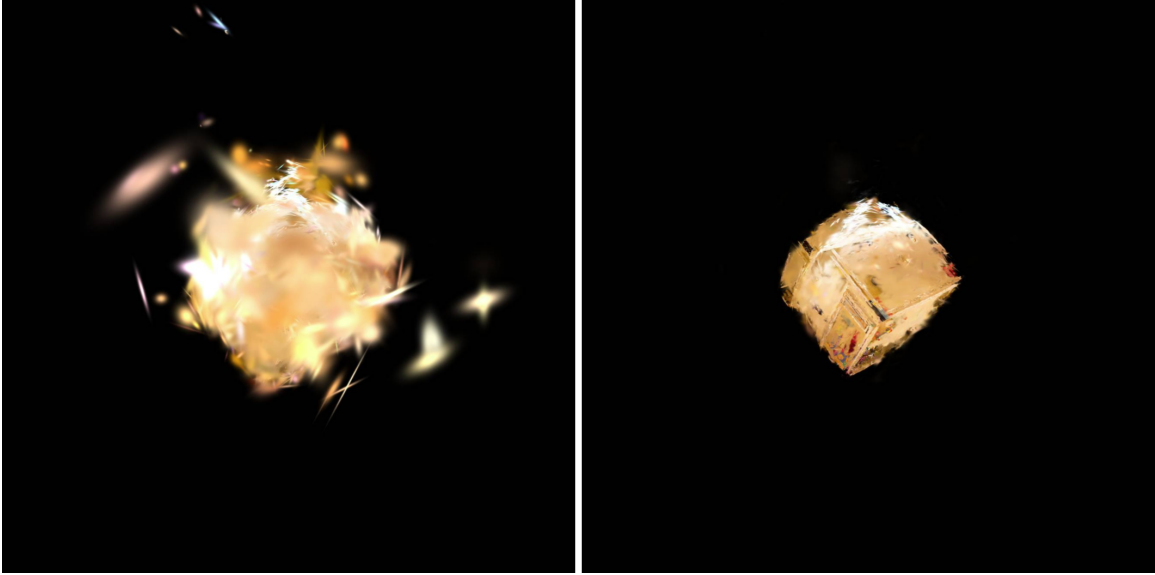


Figure 6.3: Playroom before filter (left) and after filter (right). A total of 179,399 out of 2,787,141 Gaussians were removed.

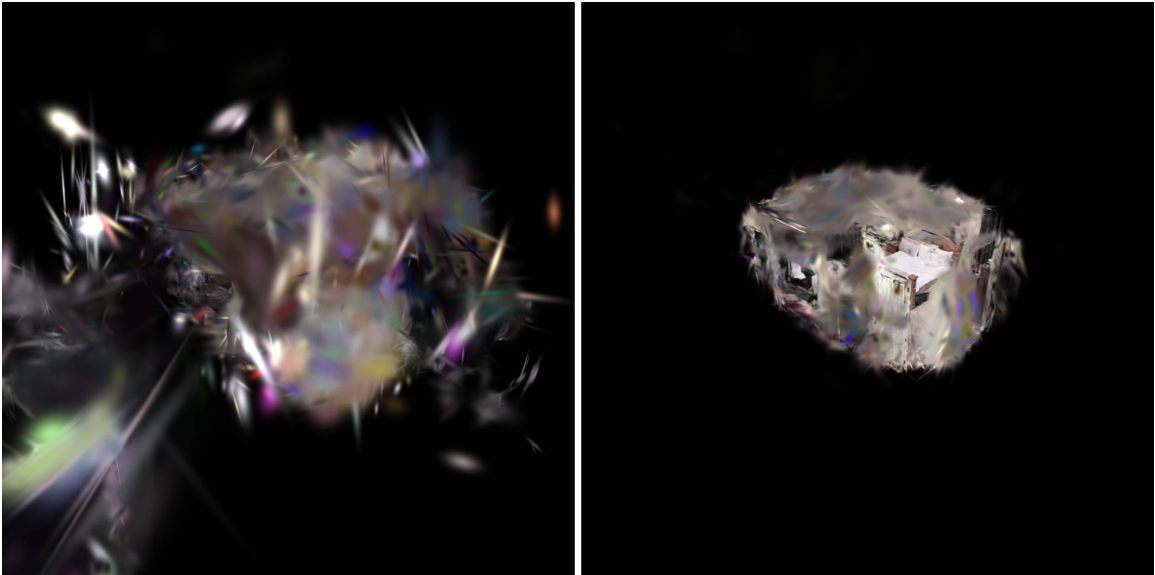


Figure 6.4: Bedroom before filter (left) and after filter (right). A total of 139,486 out of 799,249 Gaussians were removed.

As seen from Figure 6.1, 6.2, 6.3, 6.4, and Table 6.1, the filter is effective at removing the large noisy floaters that obstruct the main geometry, while also keeping real-time performance comparable to the original scene. Additionally, the filter has a fast run-time, taking < 2 minutes for each scene, including the CreepyAttic scene which contained 4,139,537 Gaussians. FPS occasionally is lower or higher than the original scene, but the proportion is small and is likely within the variance of the measurement.

6.1 Time Complexity Assessment

To assess the accuracy of the time complexity calculated in Eq. 5.1, subset scenes of CreepyAttic were created by randomly selecting 20%, 40%, 60%, and 80% of the Gaussians in the original scene. These subset scenes, along with the original scene, were run through the filter and the run-time was recorded and presented in Figure 6.5. The results indicate a linear relationship between the run-time and number of Gaussians when other parameters are frozen.

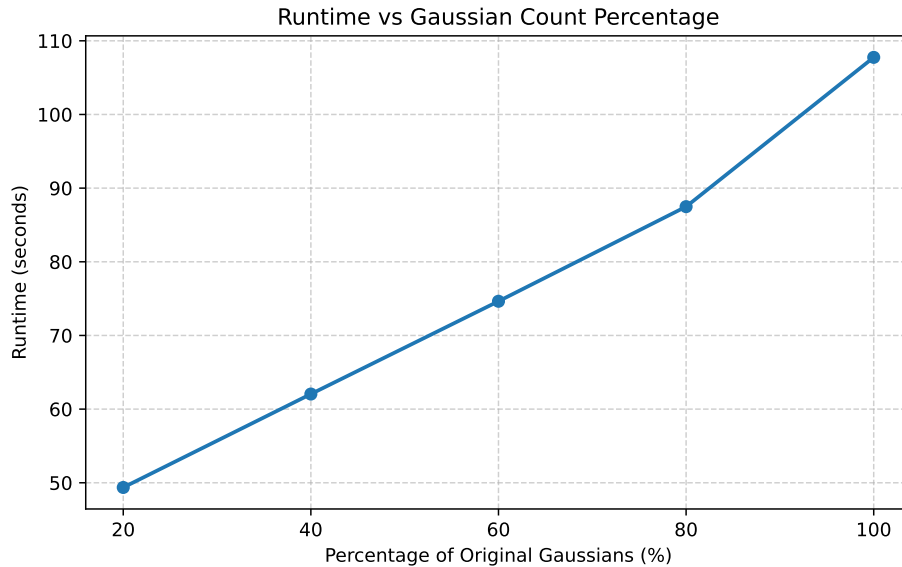


Figure 6.5: Run-time of filter on CreepyAttic scenes with varying number of Gaussians. Results indicate linear relationship between run-time and scene size.

6.2 Ablation

To better understand the contribution of each component of the filtering pipeline, an ablation study was conducted on the CreepyAttic scene. The varied parameters are the sensitivity threshold τ_{xg} , ratio threshold τ_{ratio} , ellipsoid scalars S , number of slices L , and number of cameras per slice C , while keeping $n_x = n_y = 100$ and percentile $p = 90$ fixed. A representative subset of runs is shown in Table 6.2, with the full results shown in Table A.1 in the Appendix.

The most significant factor to both the pruning rate and run-time is the number of cameras sampled, which is dictated by the number of slices, cameras per slice, and number of ellipsoids. Increasing slices and cameras per slice from 5×6 to 8×10 consistently increased pruning rate and approximately doubles or triples filtering time. Similarly, using three scalars $[1.5, 3.0, 6.0]$ instead of two $[2.0, 4.0]$ results in more aggressive pruning along with longer run-time.

In comparison, the ratio threshold τ_{ratio} and sensitivity threshold τ_{xg} had a smaller effect on pruning percentage and filtering time. In particular τ_{ratio} produced minimal variation, likely due to limited ray diversity as most Gaussians are only hit by a small number of rays, and in most cases these rays produce consistent evaluations. Due to this, the ratio of stable to unstable evaluations rarely influences the final decision for a Gaussian. However, it will be kept as a hyperparameter for the use-case where a user may increase n_x or n_y , which would result in Gaussians being sampled along many more rays.

More aggressive pruning of Gaussians is not necessarily desirable, as many of the Gaussians that were being removed in the higher pruning-percentage runs were those that meaningfully contributed to the scene. Based on these details and the ablation results, the parameters of $L \times C = 5 \times 6$, $S = [2.0, 4.0]$, $\tau_{xg} = 10^{-4}$, and $\tau_{ratio} = 0.25$

were selected as the default parameters for the filter, as it achieved a balance between accurate pruning and filtering speed.

Table 6.2: Selected runs from ablation study over filtering parameters on the CreepyAttic scene. All runs use $n_x = n_y = 100$ and PCA percentile = 0.9.

Run	x_g Thresh	Ratio Thresh	Ellipsoid Scalars	Slices $\times$ Cams	Pruned (%)	Time (s)
1	10^{-4}	0.25	[2.0, 4.0]	5×6	6.67	111.7
2	10^{-4}	0.25	[2.0, 4.0]	8×10	7.39	282.0
3	10^{-4}	0.25	[1.5, 3.0, 6.0]	5×6	7.59	160.6
4	10^{-4}	0.25	[1.5, 3.0, 6.0]	8×10	8.17	406.2
5	10^{-14}	0.25	[1.5, 3.0, 6.0]	8×10	8.86	395.2

CONCLUSION

This thesis examined the limitations of modern neural rendering techniques, focusing in particular on improving the robustness of 3D Gaussian Splatting for novel view synthesis. 3D Gaussian Splatting offers an efficient and explicit scene representation that enables real-time rendering. However, render quality tends to degrade when rendering poses that differ significantly from those seen during training. EV3DGS addressed this issue by incorporating a filter that identifies and removes unstable Gaussians from the render. This comes at the cost of additional per-frame computation which makes real-time use more difficult.

To address this trade-off, this work introduces an offline preprocessing step for Gaussian pruning, inspired by EV3DGS. By performing the sensitivity analysis as a pre-rendering stage using a set of sampled cameras rather than per-frame computation, it avoids the computational overhead during rendering. Cameras are sampled using a PCA-aligned ellipsoid to improve scene coverage during pruning. Using PCA allows the sampling ellipsoid to adapt to the shape of the scene, which helps in anisotropic environments with elongated geometry.

In addition, the sensitivity metric is calculated through a CUDA kernel within Gsplat to allow for parallel computation to keep processing times reasonable, even for complex scenes.

Overall, these contributions improve the stability of rendered views from out-of-distribution camera poses while also maintaining the real-time advantages of 3D Gaussian Splatting.

7.1 Future Work

Although this thesis was able to showcase the effectiveness in improving out-of-distribution camera pose synthesis in 3D Gaussian Splatting through the use of a pre-render filter, the implementation in this thesis is limited in speed by the lack of per-view tiling. Future work could incorporate tiling to increase the efficiency of the ray-Gaussian intersection step, as it would greatly limit the amount of Gaussians that would need to be assessed for each ray.

Another direction to pursue is a combination of online and offline filtering. This includes a pre-render step that removes globally unstable Gaussians in the scene, and a lightweight per-view filter can skip or reduce the opacity of Gaussians based on an estimated instability, instead of the fully calculated instability. This would allow for comparable real-time performance while restoring per-view adaptability.

Additionally, another approach to explore is storing instability data for certain Gaussians, and utilizing this data at run-time to allow the renderer to make more informed decisions. This can involve skipping certain Gaussians if all nearby cameras to the current camera position agree a Gaussian is unstable, or reducing the opacity of that Gaussian if there is disagreement between the neighboring cameras. This would allow for per-view adaptability while not incorporating costly ray-intersection calculations while rendering.

REFERENCES

- Bowness, D. and C. Poullis, “Extreme views: 3dgs filter for novel view synthesis from out-of-distribution camera poses”, arXiv (2025).
- Eslami, S. M. A. and D. J. Rezende, “Neural scene representation and rendering”, URL <https://deepmind.google/blog/neural-scene-representation-and-rendering/> (2018).
- Eslami, S. M. A., D. J. Rezende, F. Besse, F. Viola, A. S. Morcos, M. Garnelo, A. Ruder, A. A. Rusu, I. Danihelka, K. Gregor, D. P. Reichert, L. Buesing, T. Weber, O. Vinyals, D. Rosenbaum, N. Rabinowitz, H. King, C. Hillier, M. Botvinick, D. Wierstra, K. Kavukcuoglu and D. Hassabis, “Neural scene representation and rendering”, *Science* **360**, 6394, 1204–1210, URL <https://www.science.org/doi/abs/10.1126/science.aar6170> (2018).
- Hedman, P., J. Philip, T. Price, J.-M. Frahm, G. Drettakis and G. Brostow, “Deep blending for free-viewpoint image-based rendering”, **37**, 6, 257:1–257:15 (2018).
- Kerbl, B., G. Kopanas, T. Leimkühler and G. Drettakis, “3d gaussian splatting for real-time radiance field rendering”, URL <https://arxiv.org/abs/2308.04079> (2023).
- Khronos Group, “gluLookAt”, <https://registry.khronos.org/OpenGL-Refpages/gl2.1/xhtml/gluLookAt.xml>, openGL Reference Pages (n.d.).
- Mildenhall, B., P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi and R. Ng, “Nerf: Representing scenes as neural radiance fields for view synthesis”, in “ECCV”, (2020).
- Mittal, A., A. K. Moorthy and A. C. Bovik, “No-reference image quality assessment in the spatial domain”, *IEEE Transactions on Image Processing* **21**, 12, 4695–4708 (2012).
- Mittal, A., R. Soundararajan and A. C. Bovik, “Making a “completely blind” image quality analyzer”, *IEEE Signal Processing Letters* **20**, 3, 209–212 (2013).
- N, V., P. D, M. C. Bh, S. S. Channappayya and S. S. Medasani, “Blind image quality evaluation using perception based features”, in “2015 Twenty First National Conference on Communications (NCC)”, pp. 1–6 (2015).
- Sara Fridovich-Keil and Alex Yu, M. Tancik, Q. Chen, B. Recht and A. Kanazawa, “Plenoxels: Radiance fields without neural networks”, in “CVPR”, (2022).
- Schönberger, J. L. and J.-M. Frahm, “Structure-from-motion revisited”, in “Conference on Computer Vision and Pattern Recognition (CVPR)”, (2016).

Ye, V., R. Li, J. Kerr, M. Turkulainen, B. Yi, Z. Pan, O. Seiskari, J. Ye, J. Hu, M. Tancik and A. Kanazawa, “gsplat: An open-source library for Gaussian splatting”, arXiv preprint arXiv:2409.06765 URL <https://arxiv.org/abs/2409.06765> (2024).

APPENDIX A

FULL ABLATION RESULTS

Table A.1: Full 16 runs of the ablation study over filtering parameters on the CreepyAttic scene. All runs use $n_x = n_y = 100$ and PCA percentile = 0.9.

Run	x_g Thresh	Ratio Thresh	Ellipsoid Scalars	Slices $\times$ Cams	Pruned (%)	Time (s)
1	10^{-14}	0.01	[2.0, 4.0]	5×6	7.02	111.6
2	10^{-14}	0.01	[2.0, 4.0]	8×10	7.85	270.2
3	10^{-14}	0.01	[1.5, 3.0, 6.0]	5×6	8.21	147.1
4	10^{-14}	0.01	[1.5, 3.0, 6.0]	8×10	8.86	387.3
5	10^{-14}	0.25	[2.0, 4.0]	5×6	7.02	101.5
6	10^{-14}	0.25	[2.0, 4.0]	8×10	7.85	270.4
7	10^{-14}	0.25	[1.5, 3.0, 6.0]	5×6	8.21	160.1
8	10^{-14}	0.25	[1.5, 3.0, 6.0]	8×10	8.86	395.2
9	10^{-4}	0.01	[2.0, 4.0]	5×6	6.75	114.6
10	10^{-4}	0.01	[2.0, 4.0]	8×10	7.51	279.7
11	10^{-4}	0.01	[1.5, 3.0, 6.0]	5×6	7.71	166.5
12	10^{-4}	0.01	[1.5, 3.0, 6.0]	8×10	8.30	400.8
13	10^{-4}	0.25	[2.0, 4.0]	5×6	6.67	111.7
14	10^{-4}	0.25	[2.0, 4.0]	8×10	7.39	282.0
15	10^{-4}	0.25	[1.5, 3.0, 6.0]	5×6	7.59	160.6
16	10^{-4}	0.25	[1.5, 3.0, 6.0]	8×10	8.17	406.2